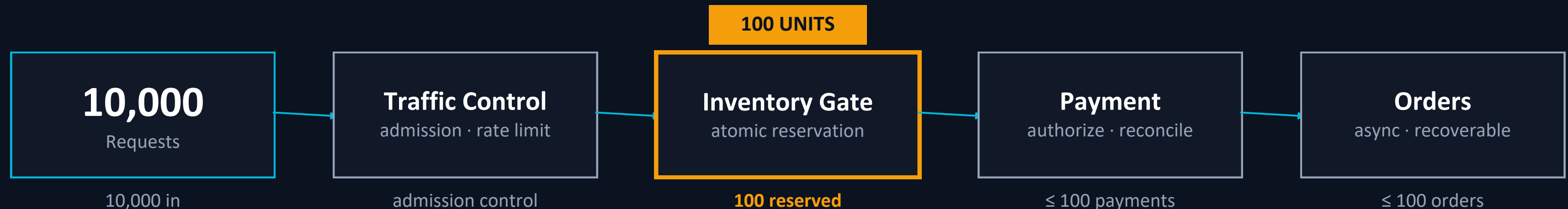


SALESTORM

High-Scale E-Commerce Flash Sale

"10,000 Customers. 100 Units. Zero Overselling."

Designing a concurrency-safe, fault-tolerant commerce platform for extreme flash-sale traffic.



TEAM MEMBERS:
OVIYA R-24AD150
SAMEERA T-24AD183
PREETHI M-24AD161

When Demand Becomes 100× the Supply

A flash sale concentrates every customer onto the same few rows of stock at the same instant.



RESULTING FAILURE MODES



- | | |
|---|---|
| 1 | Race conditions
Multiple customers read the same inventory state. |
| 2 | Overselling
More successful orders than physical stock. |
| 3 | Duplicate transactions
Repeated requests create duplicate reservations or payments. |
| 4 | Partial failures
Payment may succeed while downstream order processing fails. |
| 5 | Traffic collapse
A sudden spike can overwhelm databases and services. |

BUSINESS QUESTION

"How do we preserve transaction correctness while the traffic scales by 50×?"

Engineering Requirements

The contract the design must satisfy. Every later claim is checked against it.

FUNCTIONAL

F1	Product discovery
F2	Cart
F3	Inventory check
F4	Temporary reservation
F5	Checkout
F6	Payment
F7	Order management
F8	Shipment & fulfilment
F9	Notifications

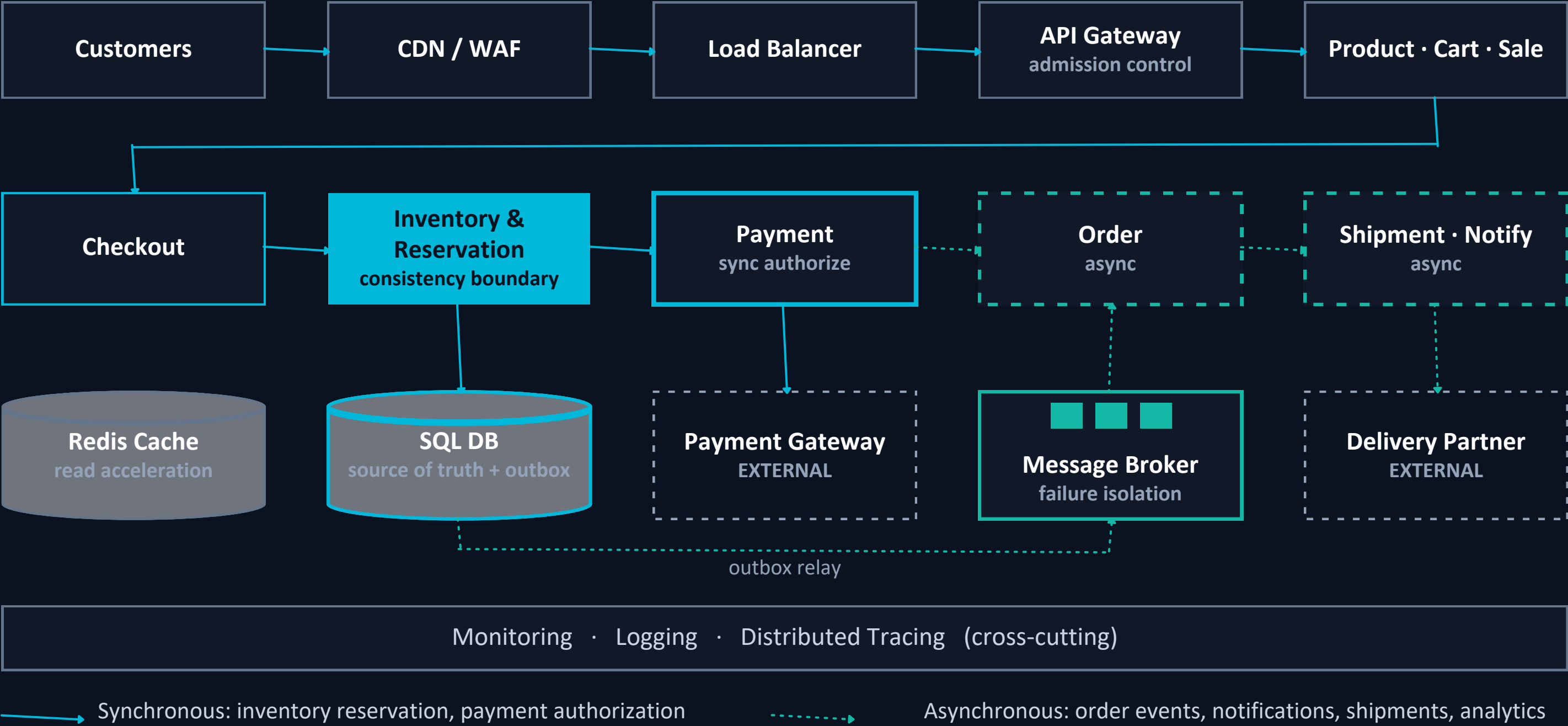
NON-FUNCTIONAL

N1	10,000 concurrent purchases
N2	500K req/s design target
N3	Zero overselling
N4	Idempotent transactions
N5	Strong inventory consistency
N6	Fault tolerance
N7	Horizontal scalability
N8	Secure payment handling
N9	Observable critical flows

SUCCESS CONTRACT	
Traffic	10,000
Stock	100 units
Max successful sales	100
Overselling	0
Duplicate payments	0
Reservation expiry	Supported
Payment/order recovery	Supported

SALESTORM — High-Level Architecture

Stateless services scale out; one path, inventory reservation, is where consistency is serialized.



Stateless services scale horizontally

Inventory Service = consistency boundary

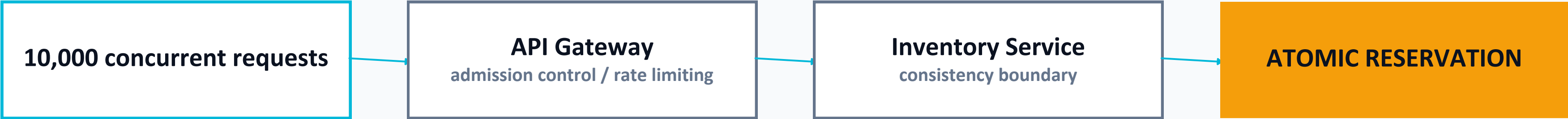
Message broker = failure isolation

Redis = read acceleration, not inventory authority

SQL = transactional source of truth

Where Overselling Is Actually Prevented

Contention is resolved in one place: a single atomic statement at the inventory boundary.

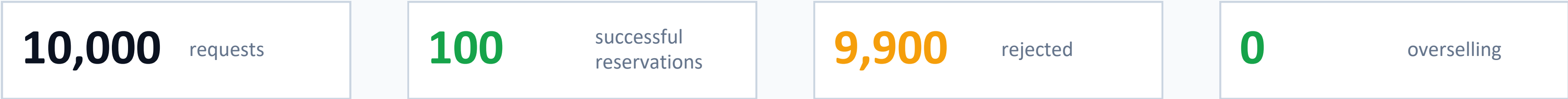


THE CORE OPERATION: ONE ATOMIC STATEMENT

```
UPDATE inventory
SET available = available - :qty
WHERE product_id = :id
  AND available >= :qty;
```

WHY IT MATTERS

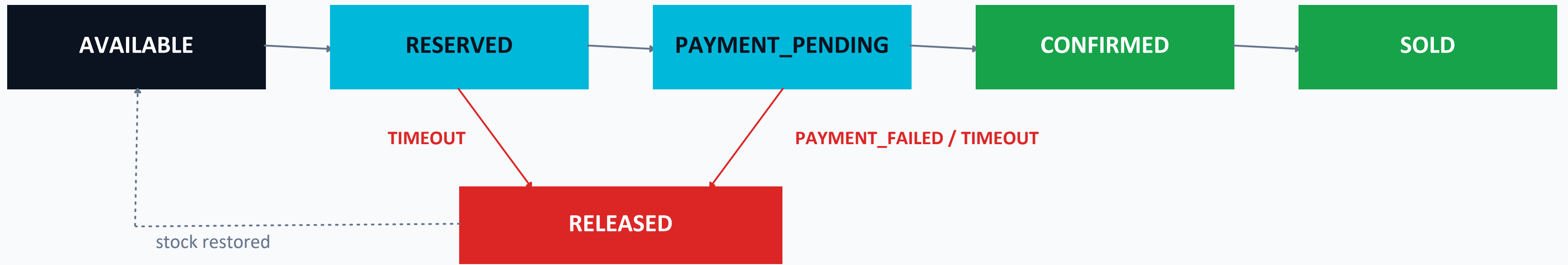
Affected rows = 1 Reservation succeeds	Naive: READ → CHECK → UPDATE Risk: race condition
Affected rows = 0 Reservation rejected	SALESTORM: ATOMIC CHECK + UPDATE Guarantee: deterministic reservation



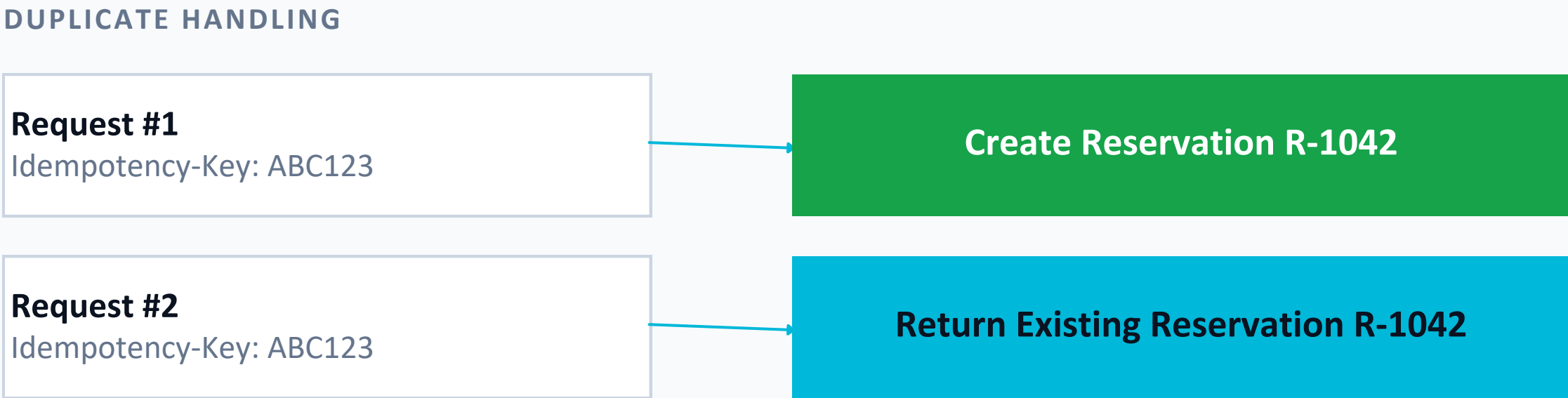
"The inventory database is not asked how much stock exists and updated later. The availability check and decrement occur as one atomic consistency operation."

Inventory Is a State Machine, Not a Counter

Every unit of stock is always in exactly one state, and every exit from a state is defined.



RESERVATION RECORD	
reservation_id	PK
product_id	FK
customer_id	FK
quantity	int
idempotency_key	UNIQUE
expires_at	timestamp
status	enum



No second reservation: **UNIQUE(idempotency_key)** enforces it.

"Idempotency converts repeated delivery of the same command into a single business effect."

Payment Success Does Not Guarantee Order Success

Money moved and order creation are two separate effects, so the design must bridge the gap.

NORMAL PATH



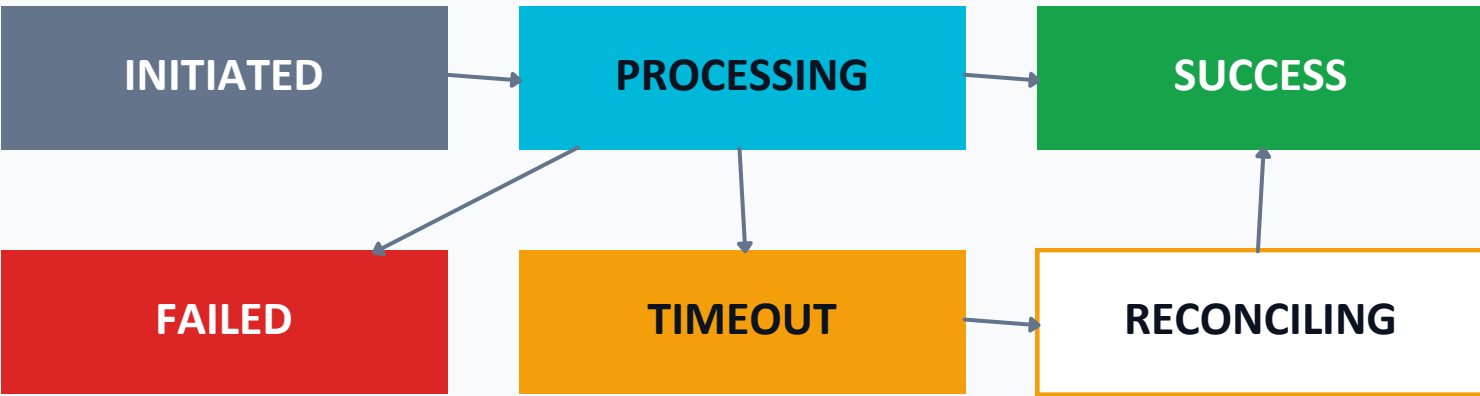
FAILURE PATH: PAYMENT SUCCEEDED, ORDER SERVICE DOWN



RELIABILITY MECHANISMS

Outbox Pattern	Event stored with the state change
Retry Policy	Bounded retries, exponential backoff
Idempotent Consumer	event_id dedupe: one business effect
Dead Letter Queue	Poison messages parked for review

PAYMENT STATES



"We do not blindly retry payment. We reconcile uncertain payment states before attempting another transaction."

Designed for Failure, Not Just the Happy Path

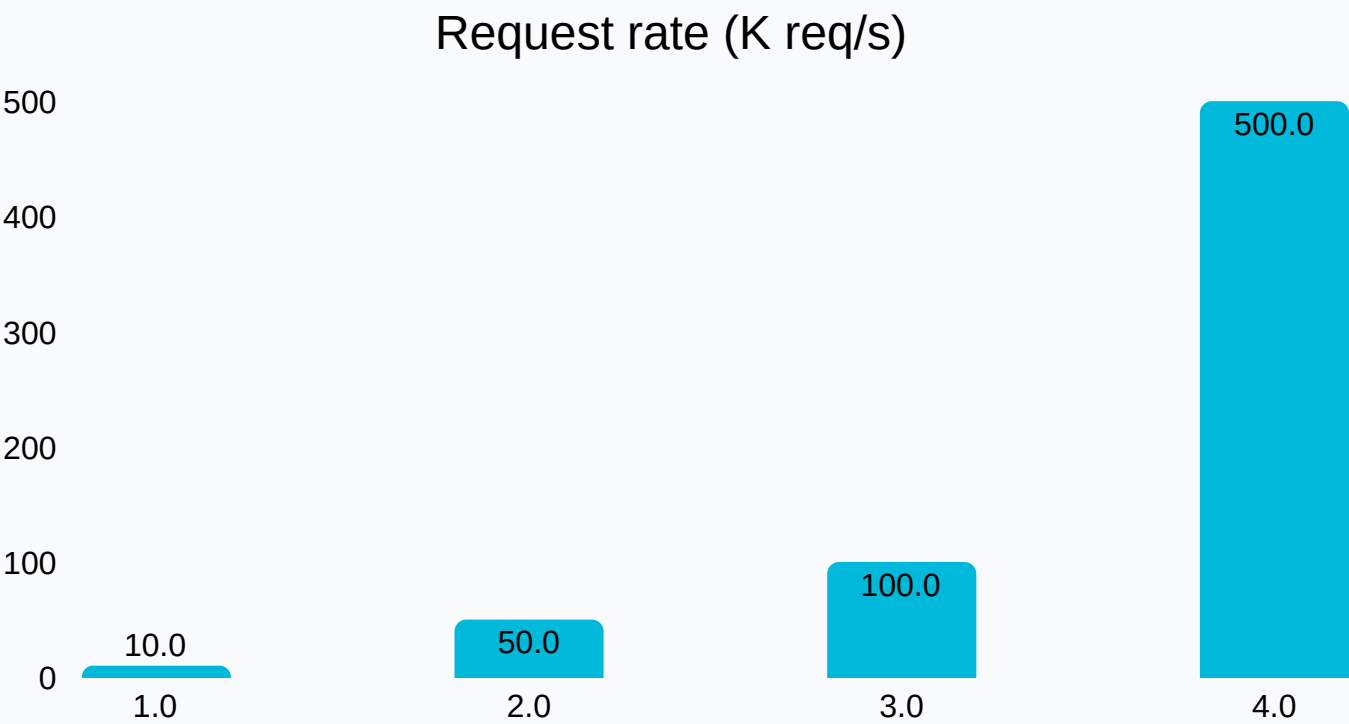
Each failure maps to an immediate response and a defined recovery.

FAILURE	IMMEDIATE RESPONSE	RECOVERY
Payment failure	Release reservation	Stock restored
Payment timeout	Reconciliation	Final gateway state
Duplicate request	Idempotency check	Existing result returned
Order Service down	Event retained	Retry after recovery
Message processing failure	Retry	Dead Letter Queue
Payment gateway unstable	Circuit breaker	Fail fast / recover
Reservation timeout	Expiry worker	Stock released
Database unavailable	Fail safely	Retry / recovery path

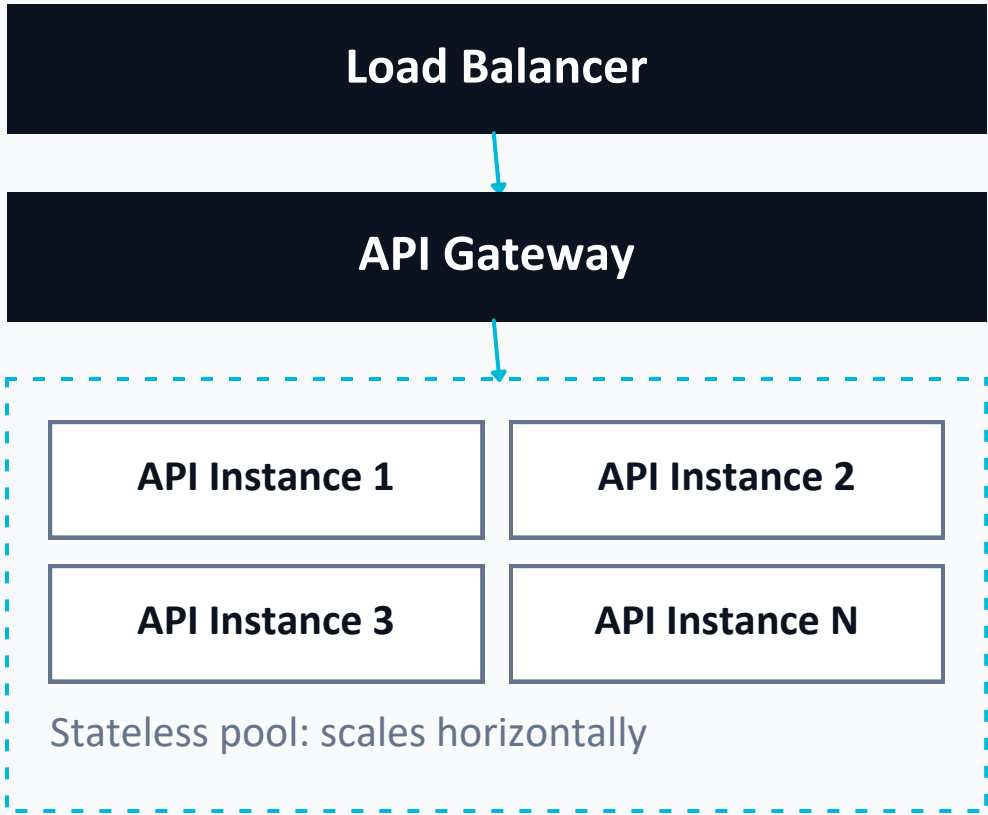
Every failure has an explicit state transition.

Scaling the System Without Scaling the Race Condition

Stateless tiers scale out freely; the hot inventory key is a contention problem, not a capacity problem.



Production scale target for architecture reasoning. Local validation: 10,000 concurrent requests.



- Redis**
read-heavy acceleration
- Message Broker**
asynchronous buffering
- SQL Cluster**
transactional persistence
- Load Balancer**
traffic distribution

THE HOT PRODUCT PROBLEM

A single flash-sale product creates concentrated contention.

Adding more API servers does not solve a hot inventory key.

- 01

Admission control
- 02

Atomic reservation
- 03

Short reservation transactions
- 04

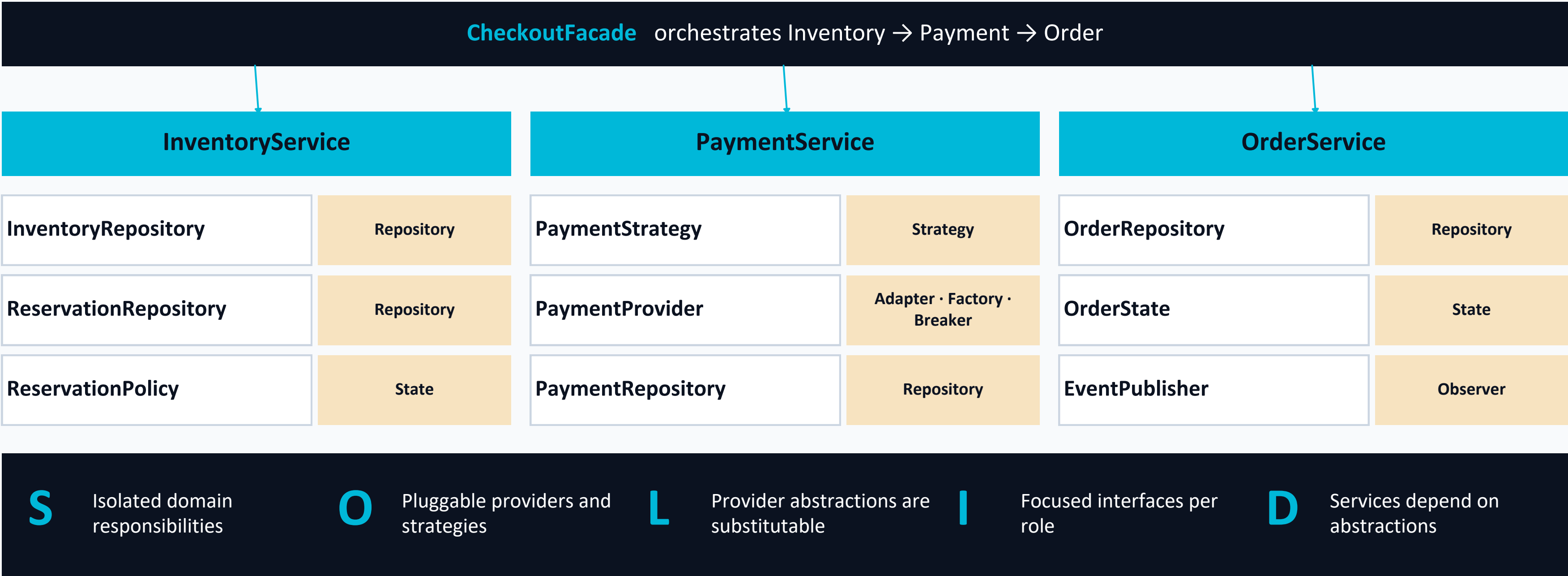
Queueing / backpressure
- 05

Stateless service scaling
- 06

Product-level partitioning where appropriate

From Architecture to Maintainable Code

Three critical modules, each isolated behind abstractions, and each pattern tied to a real problem.



Also: Factory → provider creation · Circuit Breaker → wraps external gateway calls · Observer → notifications and analytics

Correctness Must Also Be Observable

Security protects the transaction, observability proves it, validation measures it.

SECURITY

- Authentication
- Authorization
- HTTPS
- Rate limiting
- Input validation
- Secrets management
- Audit logging
- No raw card storage

OBSERVABILITY

METRICS

Request rate	P95 / P99 latency
Reservation failures	Payment failures
Queue backlog	Order conversion

LOGS

```
request_id · correlation_id
event · status · latency
```

TRACING



VALIDATION

10,000 concurrent requests

against 100 units of stock (local validation)

EXPECTED

Successful reservations	≤ 100
Oversold	0
Duplicate reservations	0

AI-ASSISTED VALIDATION

AI was used for simulation scaffolding, tests and documentation. Architecture decisions were reviewed and validated by the team.

Proof Under Pressure

The live demonstration sequence, ending in measurable results.

- 1 Launch 10,000 purchase requests
- 2 Compete for 100 units
- 3 Observe inventory reservation
- 4 Trigger duplicate request
- 5 Trigger payment failure
- 6 Payment success + Order Service failure
- 7 Recover Order Service
- 8 Verify final inventory / order state

10,000	REQUESTS
100	MAXIMUM SUCCESSFUL SALES
0	OVERSELLING
0	DUPLICATE PAYMENTS
RECOVERABLE	ORDER WORKFLOW

SALESTORM separates traffic scale from transaction correctness, allowing the platform to absorb demand without sacrificing inventory integrity.

Design First. Validate with Evidence. Defend Every Decision.